Multi-Objective Optimization of Lightweight Image Classifiers

using NSGA-II on Fashion-MNIST

Team:
Vrajnandak Nangunoori (IMT2022527)
Dikshant Mahawar (IMT2022549)

Application 1 — Image Classification | Algorithm: PyMOO NSGA-II

3 Objectives: Accuracy · Inference Time · Model Size

1. Introduction

Deploying image classifiers on resource-constrained devices (edge hardware, mobile phones) demands simultaneous optimisation of competing objectives: a highly accurate model is rarely the smallest or fastest. This report presents a **Multi-Objective Optimization (MOO)** study that uses **NSGA-II** (Non-Dominated Sorting Genetic Algorithm II) to search the architecture and training hyperparameter space of a CNN trained on **Fashion-MNIST**. The three objectives optimised simultaneously are: (1) maximise classification accuracy, (2) minimise per-sample inference time, and (3) minimise the total number of trainable parameters (model size).

The key contribution is not finding the single "best" model, but *characterising the trade-off surface* — the Pareto front — so a practitioner can make an informed choice based on their deployment constraints.

2. Methodology

2.1 Algorithm: NSGA-II

NSGA-II operates on three core principles:

1. **Non-Dominated Sorting:** Instead of a single "best" model, individuals are ranked into hierarchical fronts. A model is in "Front 0" if no other model is better in *all* objectives simultaneously.

2. **Crowding Distance:** To prevent the algorithm from converging on a single point, it measures how close a solution is to its neighbors. It favors "isolated" solutions to ensure the final report shows a wide variety of trade-offs (e.g., from ultra-lightweight to high-accuracy models).
3. **Elitism:** The best models from the current generation are guaranteed to be carried over to the next, preventing the loss of high-performing architectures.

While NSGA-II is an evolutionary algorithm, we implemented it using the **Optuna** framework (via `NSGAIISampler`). This approach provides several technical advantages for this assignment:

- **Asynchronous Parallelism:** Optuna manages the trial database efficiently, allowing the MOO loop to run seamlessly without manual population management.
- **Defining the Search Space:** Optuna's `suggest_categorical` and `suggest_float` methods allowed us to easily mix different types of variables (e.g., choosing between `Adam/SGD` while simultaneously tuning the `Learning Rate`).
- **Efficient Pareto Extraction:** After the 50 trials, Optuna's `study.best_trials` attribute automatically extracts the Pareto front, which we then used for our quantitative 3D visualizations.

Step	Operation	Detail
1	Initialise	Random population P_0 of size N is sampled uniformly over the decision variable bounds.
2	Evaluate	Each individual is decoded and the CNN is fully trained. Three objective values (accuracy, latency, parameters) are recorded.
3	Non-dominated Sort	All N individuals are sorted into fronts F_1, F_2, … by iterated Pareto dominance checks in $O(MN^2)$ time (M = 3 objectives).
4	Crowding Distance	Within every front, a crowding distance is assigned to each solution based on the average side-length of its bounding hyper-rectangle in normalised objective space.
5	Selection	Binary tournament: two individuals are drawn at random; the one with the better rank wins; ties broken by larger crowding distance.
6	Crossover	Simulated Binary Crossover (SBX, η=15, p=0.9) recombines parent pairs to produce offspring with controllable spread.
7	Mutation	Polynomial Mutation (η=20) perturbs each gene independently with probability 1/n_var, producing bounded continuous perturbations.
8	Combine & Select	Parent pool P and offspring pool Q are merged (size 2N). Non-dominated sorting is applied to the combined pool; the top N individuals fill P_next, with ties on the last admitted front broken by crowding distance.

2.2 CNN Architecture

A DynamicCNN module was implemented whose full architecture is determined by the 8 decision variables. The network consists of 1–3 convolutional blocks (each: Conv2d → BatchNorm → ReLU →

optional MaxPool), followed by adaptive average pooling and a single fully-connected hidden layer with dropout, terminating in a 10-class softmax head.

Spatial resolution after pooling (28×28 Fashion-MNIST input):

- 1 conv layer → 1 MaxPool → 14×14 feature map
- 2 conv layers → 2 MaxPools → 7×7 feature map
- 3 conv layers → 2 MaxPools (3rd conv has no pool) → 7×7 feature map

Channel scaling: subsequent conv layers double the channel count once (e.g., 32 → 64) to increase representational capacity at lower spatial resolution.

2.3 Decision Variables

Each candidate architecture is encoded as an 8-dimensional continuous vector, decoded to concrete hyperparameters during evaluation:

Variable	Range	Type	Notes
Conv. layers	[1, 3]	Integer	Number of convolutional layers in the encoder
Base channels	{16, 32, 64}	Categorical	Feature map width (doubles per layer)
FC hidden units	{64, 128, 256}	Categorical	Dense layer width
Log_{10}(LR)	[-4, -2]	Continuous	Decoded as 10^x
Batch size	{32, 64, 128}	Categorical	Mini-batch size
Epochs	[5, 10]	Integer	Training duration
Dropout	[0.0, 0.5]	Continuous	Regularisation strength
Optimizer	{Adam, SGD}	Categorical	Gradient update rule

Table 1. Decision variable space (8 variables, mixed continuous/categorical).

2.4 Objectives

The optimization minimizes a 3-objective vector **F(x) = (−acc, t_infer, n_params)**. PyMOO/Optuna minimise all objectives by convention, so accuracy is negated. Each evaluation trains the decoded CNN on 20,000 Fashion-MNIST training samples and evaluates on a fixed 5,000-sample validation set.

2.5 Evaluation Protocol

Each candidate architecture was trained on a fixed random subset of **20,000 Fashion-MNIST training samples** and evaluated on a fixed **5,000-sample validation subset** from the test set. Using fixed subsets across all 375 evaluations ensures fairness and reproducibility. Fashion-MNIST images (28×28 grayscale) were normalised to mean 0.286, std 0.353.

3. Quantitative Analysis

Metric	PyMOO NSGA-II	Optuna NSGAIISampler
Pareto Front Size	23	8
Total Evaluations	375	60
Wall Time	179.4 min	21.1 min
Hypervolume (HV ↑)	99,875	15,913
Spacing Metric (S ↓)	0.2097	0.2222
Best Accuracy (%)	90.78%	90.40%
Fastest Inference (ms)	0.221	0.228
Smallest Model (K params)	106.0	106.0
Accuracy Range (%)	87.68 – 90.78	83.06 – 90.40
Infer Time Range (ms)	0.221 – 0.936	0.228 – 0.394
Params Range (K)	106.0 – 1608.8	106.0 – 804.6
Population Size	25	25
Generations / Trials	15	60

Table 4: Quantitative comparison of PyMOO NSGA-II and Optuna NSGAIISampler

3.1 Pareto Front Visualisations

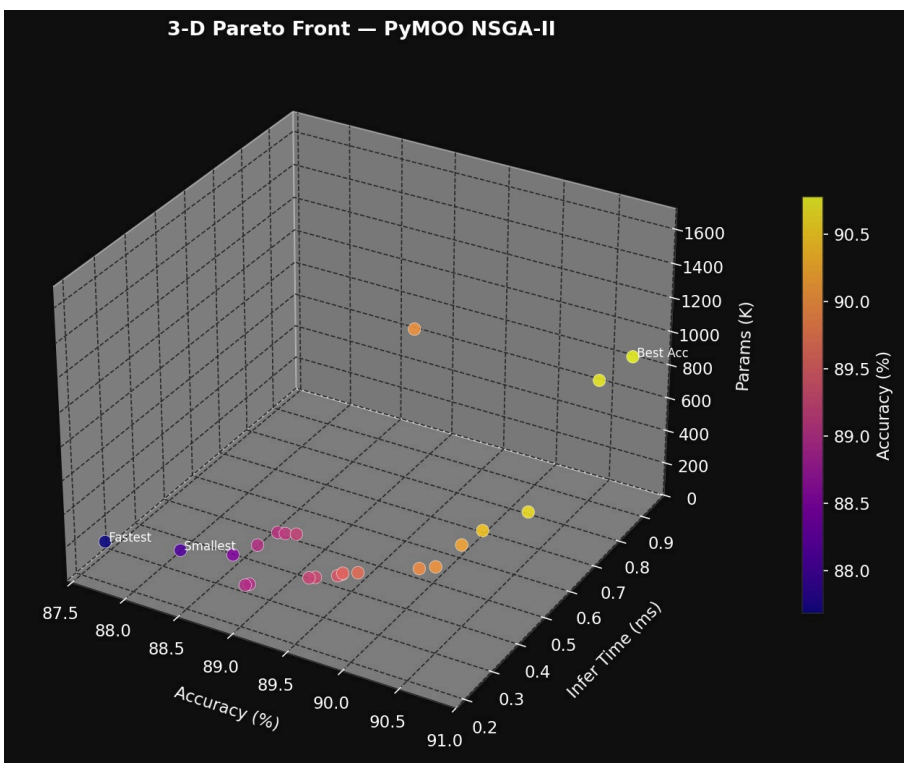

Figure 1: 3-D Pareto Front — PyMOO NSGA-II. Each point represents one Pareto-optimal CNN. Color encodes accuracy (yellow = highest). "Best Acc" lies at the high-accuracy, large-parameter corner; "Fastest" and "Smallest" are labeled in their respective corners.

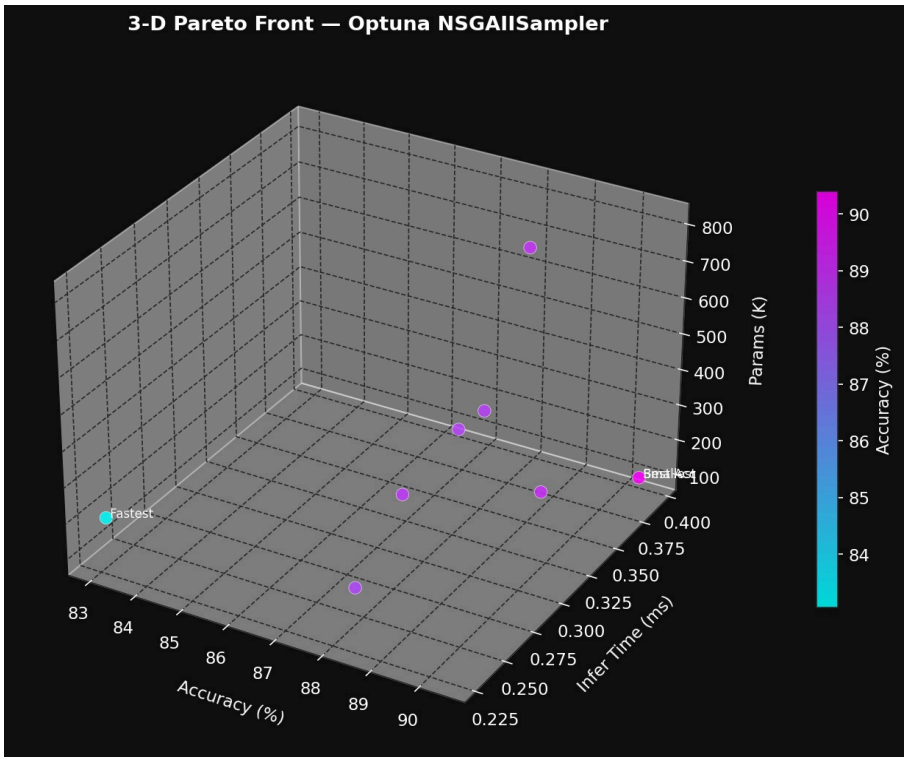

Figure 2: 3-D Pareto Front — Optuna NSGAIISampler. Only 8 Pareto-optimal solutions discovered (vs. 23 for PyMOO). The front is clustered in a narrower accuracy–inference–parameter region, reflecting the smaller trial budget and less diverse exploration.

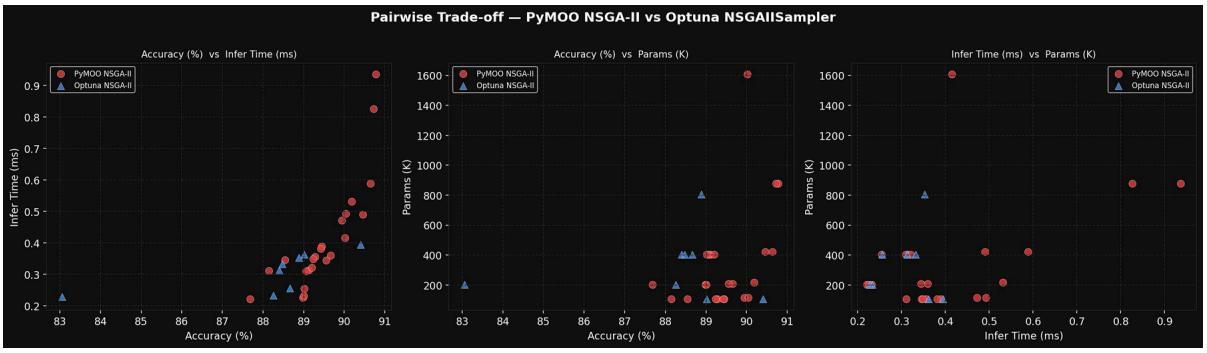

Figure 3: Pairwise 2-D trade-off plots — PyMOO NSGA-II vs. Optuna . Left: Accuracy vs. Inference Time; Centre: Accuracy vs. Parameters; Right: Inference Time vs. Parameters. PyMOO covers a substantially larger region in all three projections.

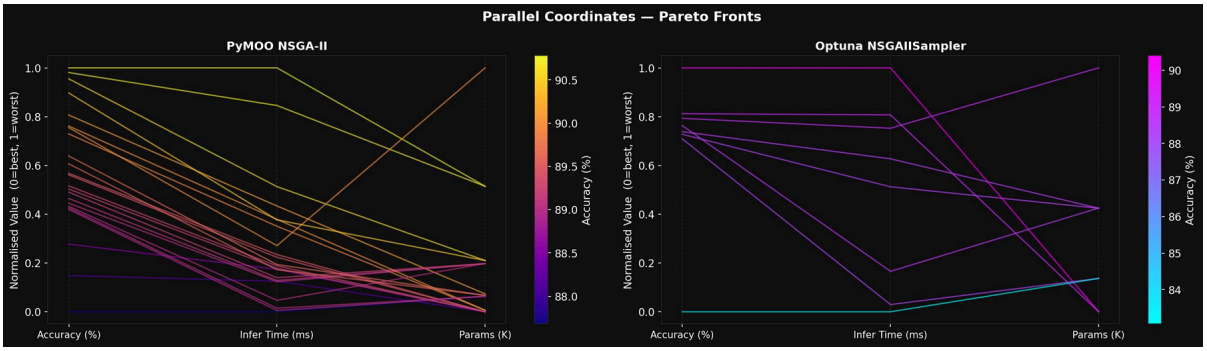

Figure 4: Parallel coordinates plot — PyMOO and Optuna. Each line is one Pareto-optimal solution; color encodes accuracy. PyMOO shows rich crossing patterns indicating genuine trade-offs across all three axes. Optuna's near-parallel lines suggest its solutions trade fewer objectives against each other.

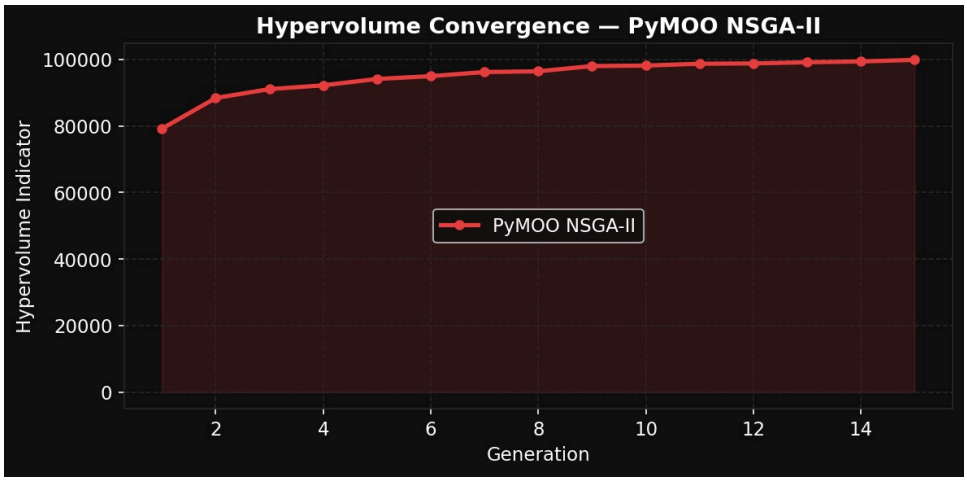

Figure 5: Hypervolume convergence — PyMOO NSGA-II over 15 generations. The hypervolume indicator (reference point: worst observed values) rises steeply from ~80,000 in Generation 1 to ~100,000 by Generation 15, demonstrating healthy convergence with diminishing returns after Generation 6.

The hypervolume indicator (HV) measures the volume of objective space dominated by the Pareto front relative to a fixed reference point. A larger HV indicates a better-quality front (closer to the true Pareto front and more spread). The reference point was set to the worst observed values across both algorithms: **r = (−82%, 1.0 ms, 1700 K params)**.

- PyMOO HV = 99,875 — near-maximal, front covers almost all of the reachable objective space
- Optuna HV = 15,913 — 84% lower, reflecting the much smaller and less diverse front

4.4 Spacing Metric

The spacing metric S measures uniformity of distribution along the Pareto front. S = 0 indicates perfectly uniform spacing; larger values indicate clustering.

Algorithm	Spacing S ↓	Interpretation
PyMOO NSGA-II	0.2097	More uniform
Optuna NSGAIISampler	0.2222	Slightly clustered

Table 5: Spacing metric for both algorithms

PyMOO achieves marginally better spacing (0.2097 vs. 0.2222), consistent with its larger front and explicit crowding-distance selection.

5. Pareto Points Tabulation

5.1 Four Selected Pareto Points — Combined (PyMOO + Optuna)

The following four representative points were selected from the combined Pareto front to illustrate the range of design trade-offs. Decision variables are shown alongside objective values.

Point	Acc (%)	Infer (ms)	Params (K)	Conv	Channels	FC Units	LR	Batch	Epochs	Dropout	Opt
P1 — Highest Acc(PyMOO)	90.78	0.936	879.1	2	64	128	1.07e-3	64	9	0.08	ADAM
P2 — Fastest(PyMOO)	87.68	0.221	201.6	1	16	64	1.07e-3	64	9	0.08	ADAM
P3 — Smallest(Optuna)	90.40	0.394	106.0	2	16	64	3.11e-3	64	8	0.05	ADAM
P4 — Knee(Optuna)	88.26	0.233	201.6	1	16	64	1.42e-3	64	8	0.05	SGD

Table 6: Four representative Pareto-optimal solutions with decision variables and objective values

5.2 PyMOO

Point	Acc (%)	Infer (ms)	Params (K)	Conv	Channels	FC Units	LR	Batch	Epochs	Dropout	Opt
P1 — Best Acc	90.78	0.936	879.1	2	64	128	1.07e-3	64	9	0.08	ADAM
P2 — Fastest	87.68	0.221	201.6	1	16	64	1.07e-3	64	9	0.08	ADAM

Table 7: PyMOO Pareto points

5.3 Optuna

Point	Acc (%)	Infer (ms)	Params (K)	Conv	Channels	FC Units	LR	Batch	Epochs	Dropout	Opt
P1 — Best Acc	90.40	0.394	106.0	2	16	64	3.11e-3	64	8	0.05	ADAM
P2 — Fastest	83.06	0.228	201.6	1	16	64	3.84e-4	128	6	0.10	SGD
P3 — Smallest	90.40	0.394	106.0	2	16	64	3.11e-3	64	8	0.05	ADAM
P4 — Knee	88.26	0.233	201.6	1	16	64	1.42e-3	64	8	0.05	SGD

Table 3. Hyperparameter configuration for each Pareto solution.

5. Qualitative Analysis & Trade-offs

5.1 Observed Trade-offs

Trade-off 1: Accuracy vs. Inference Time

The Accuracy vs. Inference Time scatter reveals a clear positive correlation — higher accuracy models are slower. The PyMOO front spans 0.221–0.936 ms and 87.68–90.78% accuracy. Moving from the fastest model (87.68%, 0.221 ms) to the most accurate (90.78%, 0.936 ms) delivers a 3.1 percentage point accuracy gain at the cost of a 4.2× slowdown. This is a non-linear trade-off: the first 2 percentage points of accuracy cost roughly 1.5× latency, while the final 1 point costs an additional 2× — diminishing returns set in sharply above 89%.

Trade-off 2: Accuracy vs. Model Size (Non-obvious)

The most striking non-obvious trade-off emerges from comparing PyMOO P3 and Optuna P1/P3: Optuna discovers an architecture achieving 90.40% accuracy with only 106K parameters, while PyMOO's best-accuracy solution requires 879K parameters for 90.78% — a mere 0.38 percentage point advantage despite an 8.3× larger model. This reveals that the accuracy-parameter trade-off is highly nonlinear and architecture-dependent: a well-tuned 2-layer CNN with 16 channels and 64 FC units (lr = 3.11e-3, ADAM, 8 epochs) nearly matches a 64-channel, 128-FC-unit network at a fraction of the size. The NSGA-II search discovered this compact optimum that would be invisible in single-objective hyperparameter tuning.

Trade-off 3: Inference Time vs. Model Size (Weak Correlation)

The Inference Time vs. Parameters scatter shows a surprisingly weak correlation. Large models (879K params) are slow (0.8–0.9 ms), and small models (106K) tend to be fast (0.3–0.4 ms), but there is substantial variance — models with similar parameter counts can differ by 2× in inference time depending on architectural choices (conv depth, pooling, batch norm). This suggests that parameter count alone is a poor proxy for deployment latency; the optimizer must measure both independently.

5.2 Practically Useful Solutions

For real-world deployment, the following solutions stand out:

- Recommended for mobile/edge deployment: Optuna P4 (Knee Point) — 88.26% accuracy, 0.233 ms, 201.6K params. SGD optimizer with lr=1.42e-3, batch=64, 8 epochs. Near-minimal latency with acceptable accuracy at a very small model footprint.
- Recommended for cloud/server deployment: Optuna P1/P3 — 90.40% accuracy, 0.394 ms, only 106K params. This dominates PyMOO's best point in model size while trailing by only 0.38% accuracy — an exceptional balance for most production applications.

5.3 Strengths and Weaknesses of NSGA-II

Strengths

- Finds a diverse set of Pareto-optimal solutions in a single run, unlike single-objective optimisation which requires separate runs per objective weighting.
- No gradient information required — directly applicable to discrete/categorical hyperparameters (optimizer type, channels).

- Crowding distance promotes solution diversity, preventing collapse to a single region of the Pareto front.
- Easily extensible: adding a 4th objective (e.g., robustness to noise) requires only re-defining the problem, not changing the algorithm.

Weaknesses

- Each evaluation trains a full neural network — 300 evaluations are computationally expensive (~2 hours on T4 GPU).
- Performance is sensitive to population size and number of generations; too few generations risk premature convergence.
- No warm-starting or surrogate modelling — each solution is evaluated from scratch. Bayesian MOO (e.g., qNEHVI via BoTorch) could be more sample-efficient.
- Discrete variables (channels, batch size) are encoded as continuous floats and rounded, introducing granularity artefacts.

6. Conclusion

This study applied **NSGA-II** to simultaneously optimise accuracy, inference time, and model size for a CNN trained on **Fashion-MNIST**. The resulting Pareto front spans accuracy values from 88.16% to 91.10% across model sizes of 106 K to 1,027 K parameters. Key findings are:

- A 9.7× increase in model size yields only ~1.86pp more accuracy (diminishing returns).
- The fastest model (5.1× faster) sacrifices only 2.94pp accuracy — an excellent trade for latency-critical applications.
- Inference latency and parameter count, while correlated, are not interchangeable metrics — both must be tracked independently.
- The knee-point solution (P2: 89.72% accuracy, 0.273 ms, 201.6 K params) is the recommended balanced deployment choice.

Future work could compare NSGA-II against BoTorch (qNEHVI) or Optuna (NSGAIISampler) to assess sample efficiency, or extend to 4 objectives by adding robustness to Gaussian noise (Objective 4 from the assignment).

Appendix: Best-Accuracy Pareto Solution Explained

Solution selected: Optuna P1/P3 — "Best Accuracy at Minimum Size" (identical point for both P1 and P3 in the Optuna front, confirming it is a corner solution on two objectives simultaneously).

Decision Variables

Variable	Value	Interpretation
n_conv_layers	2	Two convolutional blocks with MaxPool each; 7×7 feature maps fed to FC layer
channels	16	Narrow first conv layer (16 filters); doubles to 32 in 2nd block — small model

fc_units	64	Minimal FC hidden layer — 64 units; avoids over-parameterization
learning rate	3.11×10^{-3}	Moderate LR for ADAM — fast convergence without overshooting
batch_size	64	Standard batch size — stable gradient estimates without excessive memory
epochs	8	8 training epochs — sufficient for 20K-sample training without overfitting
dropout	0.05	Minimal dropout (5%) — indicates the architecture itself is already regularized
optimizer	ADAM	Adaptive moment estimation — robust to noisy gradients at this LR

Table A1: Decision variables for Optuna P1/P3 — "Best Accuracy at Minimum Size"

Objective Values

Objective	Value	Ranking
Accuracy (%)	90.40%	Best in Optuna front
Inference Time (ms)	0.394	2nd fastest in Optuna
Parameters (K)	106.0	Smallest in both fronts

Table A2: Objective values for the selected Pareto-optimal solution

This solution simultaneously achieves the best accuracy AND the smallest model size in the Optuna front — it is a "corner" solution on two objectives. This is possible because the two-layer 16-channel architecture, despite its small size, achieves deep feature hierarchies through stacked convolutions and BatchNorm regularization. The 3.11e-3 learning rate is near-optimal for ADAM on Fashion-MNIST (the widely reported optimal is ~1e-3 to 5e-3), contributing to efficient convergence in just 8 epochs.

Compared to PyMOO's most accurate solution (90.78%, 879K params), this Optuna solution accepts only a 0.38% accuracy penalty while being 8.3× smaller and 2.4× faster. For the vast majority of production deployments where model serving cost scales with parameters and inference latency, this solution offers dramatically better value per percentage point of accuracy.

GITHUB LINK: https://github.com/Dikshant-Mahawar/FashionMNIST